Project 15: Password Strength Analyser

Student name: Khyhael jones

Student ID: 97667623

Lecturer: Ehsan Nowroozi

Cybersecurity BSc Year 3

Contents

Project 15: Password Strength Analyser 1

Abstract 1

Introduction 2

Length Analysis 2

Character Variety Analysis 3

Dictionary-Based Analysis 3

User Interaction 3

Technical Implementation 3

Conclusion 5

Abstract

This project implements a Password Strength Analyzer a program built in the Python programming language that evaluates security of passwords and decides how strong or weak some passwords are. The software normally considers the length of the password and the character variety to withstand the already conducted variants of attacking dictionary-based passwords. A password is awarded a score out of six and is either classified as weak, moderate, or strong with recommendations on how to enhance its security features being offered.

To satisfy the need of passwords consisting of a composition of distinctive character types, for example, uppercase letters, lowercase letters and even special characters, regular expressions are used. To find out the weaknesses caused by the user's choose password, the password entered is matched against a list of easily guessed passwords.

The code is constructed in such a way that it can be used interactively, and passwords can be typed in for the users to evaluate how they measure up. To make it easy to integrate into a variety of different test scenarios such as Google Colab, this application includes support for an in-memory dictionary.

This project aims at creating awareness towards the usage of strong passwords by employing passwords security concepts as applied in the case studies conducted among predecessors. It is ideal

for education, personal use or even to provide the basis for the cheaper internal solution that comes for more sophisticated password management systems. Providing ac

Introduction

Anyone owning a computer today knows the importance of strong passwords, and unfortunately cyber criminals have access to millions of devices that have been secured with neither a strong password, nor a password entirely. This endless problem can now be overcome with the Positive strength analyser, a tool that is based on python and is used to check password vulnerabilities comprehensively.

The program fuses qualitative and quantitative measures in attempting to produce an approximate figure of how strong the desired password is. Password length, types of characters, and the dictionary attack potential using a list of probable passwords are assessed. Consequently, a mark is given, and the password is classified as weak, moderate, or strong with recommendations to rectify weaknesses.

The strength analyser is efficient and even beginner programmers can achieve optimal results, making it great for individuals and as an introductory module in bigger password security systems. Moving forward, the tool will aid users on how to create and set up strong password architecture and resilience in managing accounts and will subsequently ease up breaches in the technologically advancing world.

The tool is encapsulated within a class known as PasswordStrengthAnalyzer and upon creation takes an optional parameter being a dictionary having an arbitrary amount of commonly weak passwords. This parameter allows the program to raise an alarm that the password is weak, and the chances of a successful attack being conducted is high. The core functionalism is provided by the analyse method, through which three key aspects can be measured against passwords:

Length Analysis

Password length is one of the most crucial factors for security. The longer the password, the greater the number of configurations that can be achieved and the more effective the brute-force attack will be. Such a tool distributes passwords into three levels of security:

```
# 1. Length check
if len(password) >= 12:
    score += 2
elif len(password) >= 8:
    score += 1
else:
    issues.append("Password is too short.")
```

- Strong: twelve or more characters (2 points).
- Acceptable: 8–11 characters (1 point).
- Weak: Less than 8 characters.

Character Variety Analysis

The use of different characters increases a password's strength which makes it difficult for an attacker to guess Leverage The tool checks for the presence of:

- Uppercase letters.
- Lowercase letters.
- Digits.
- Special characters (e.g., @, #,!).

Each category adds one point to the password's score if present.

```
# 2. Character variety check
if re.search(r'[A-Z]', password):
    score += 1
else:
    issues.append("Add uppercase letters for better strength.")

if re.search(r'[a-z]', password):
    score += 1
else:
    issues.append("Add lowercase letters for better strength.")
```

```
# 3. Dictionary-based check
if password.lower() in self.dictionary:
    issues.append("Password is a common word found in the dictionary.")
```

```
if re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
    score += 1
else:
    issues.append("Add special characters for better strength.")
```

assword" and ssword dictionaries and -time basis. Over the command-line interface, they can also enter relationships and await the corresponding results. The results consist of a qualitative assessment, three in total: weak, moderate, and strong, the score is on a scale of six, and certain aspects where they can improve. For example:

```
Enter a password to analyze (or 'exit' to quit):
Password: happy123456_
Password Strength: Moderate
Score: 4/6
Recommendations to improve your password:
  - Add uppercase letters for better strength.
  - Add special characters for better strength.

Enter a password to analyze (or 'exit' to quit):
Password:
```

Moreover, the encouragement that this password tool is very interactive partly helps the users to be able to select the stronger ones at the end of the overall password selecting process.

Technical Implementation

The application employs the re module in Python programming language to match patterns, which is crucial in verifying the presence of certain types of characters in this instance:

```
if re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
    score += 1
else:
    issues.append("Add special characters for better strength.")
```

This code proceeds to check if the password has at least one capital letter and subsequently, increases the points score. Further, the use of the python set data structure enhances the lookup.

operation needs during the dictionary check, thus improving the scalability and efficiency.

```
else:
    issues.append("Add uppercase letters for better strength.")

if re.search(r'[a-z]', password):
    score += 1
else:
    issues.append("Add lowercase letters for better strength.")

if re.search(r'[0-9]', password):
    score += 1
```

Evaluation and Rating System

According to the evaluation metrics, each password is assigned a score by the analyser. The total score reflects the extent of the password's strength:

- Strong (6 points): All security criteria are met, and it is fully secure.
- Moderate (4–5 points): The password is satisfactory, but it can be improved.
- Weak (<4 points): The password is insecure and needs considerable improvement.

```
# Strength rating
if score >= 6:
    rating = "Strong"
elif score >= 4:
    rating = "Moderate"
else:
    rating = "Weak"

return {
    "rating": rating,
    "score": score,
    "issues": issues
```

It is fair to say that the scoring system is simple. However, it is beneficial, as it is effective in showing what actions should be undertaken to the passwords.

Applications and Use Cases

There is a Password Strength Analyzer that is multifunctional, and can be used in the following areas:

- Educational: Teaching students and professional the rules of how to create and manage secure passwords.
- Personal use: Helping the users in assessing and enhancing their password.
- System use: It acts as a part of password management systems or processes of user authentication.

The tool helps to raise awareness and thus stress the need for action in terms of securing oneself, and therefore helps to achieve the overall objective of enhancing the digital hygiene.

Limitations and Future Improvements

Nonetheless, the present system has always certain shortcomings and these are pointers on the user end which include the following:

- Entropy Measurement: The system does not carry out an entropy measure in a mathematical sense which would have improved the accuracy in determining the strength of the passwords.
- Static Dictionary: A simple dictionary that had several weak passwords was created and you must manually update for it to be useful.
- Real-Time Feedback: The system did not provide an interactive real-time interaction where feedback was provided while auto-generated passwords were being typed but instead ran in a batch mode.

To try and solve these issues, the following improvements suggest themselves for future development:

- Dynamic – entropy construction based on the Shannon entropy formula would address the problems.
- Weak password databases which are available over the internet could be incorporated into the system and as such they would be updated automatically.
- A graphical user interface (GUI) could be produced to increase accessibility.

Conclusion

The Password Strength Analyzer is a useful and efficient solution when it comes to measuring the security of a password. By considering length, number of characters and even the dictionary check, much wider view is given, and constructive feedback is provided to users. Even though there are some constraints, the tool can sufficiently address the uncertainties around practical acceptance of adequate password practice. With more improvements, it can be integrated into personal and organization level password security management software. This project emphasizes the need of elevating personal password cleanliness to manage its risks and protect IP networked worlds today.

Bibliography

Books and Articles

Silver, D., Jana, S., Boneh, D., Chen, E., & Jackson, C. (2014). Password Managers: Attacks and Defenses. 23rd USENIX Security Symposium (USENIX Security 14), San Diego, CA, pp. 449–464. Available at: https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/silver [Accessed 2 Dec. 2024].

Weber, J. E., Guster, D., Safonov, P., & Schmidt, M. B. (2008). Weak Password Security: An Empirical Study. Information Security Journal: A Global Perspective, 17(1), pp. 45–54. Available at: https://doi.org/10.1080/10658980701824432 [Accessed 2 Dec. 2024].

Malone, D., & Maher, K. (2012). Investigating the Distribution of Password Choices. Proceedings of the 21st International Conference on World Wide Web (WWW '12), New York, NY, USA, pp. 301–310. Available at: https://doi.org/10.1145/2187836.2187878 [Accessed 2 Dec. 2024].

Zviran, M., & Haga, W. J. (1999). Password Security: An Empirical Study. Journal of Management Information Systems, 15(4), pp. 161–185. Available at: https://doi.org/10.1080/07421222.1999.11518226 [Accessed 2 Dec. 2024].

Houshmand, S., Aggarwal, S., & Flood, R. (2015). Next Gen PCFG Password Cracking. IEEE Transactions on Information Forensics and Security, 10(8), pp. 1776–1791. Available at: https://doi.org/10.1109/TIFS.2015.2428671 [Accessed 2 Dec. 2024].

Online Resources

Miessler, D. (2024). 10 Million Password List - Top 1,000,000. GitHub SecLists. Available at: https://github.com/danielmiessler/SecLists/blob/master/Passwords/Common-Credentials/10-million-password-list-top-1000000.txt [Accessed 2 Dec. 2024].